

A Single Chain Suffices: Re-examining Diversity-Enhanced flowMC Ensembles under Equal Sampling Budgets

Mingyu Shi¹

¹Independent Researcher

Abstract

Normalizing-flow Markov chain Monte Carlo (MCMC), of which flowMC [1,2] is a representative implementation, adds a learned global proposal on top of local moves so that difficult target densities become easier to sample. A natural idea to make such a sampler more reliable is to run several of them with different architectures, treat them as an ensemble, and pool their samples. This paper started as an attempt to build and validate exactly this kind of method: a five-member diverse flowMC ensemble equipped with three aggregation mechanisms: ESS-based adaptive tempering, coverage-aware reweighting, and quality-based member weighting. On six benchmark targets in 20 and 50 dimensions, the ensemble seemed to reduce the average marginal Jensen–Shannon (JS) distance by about 18% compared with a single flowMC chain, and the effect looked large and statistically significant everywhere. We then ran one control that changed the whole story. The equal-per-member ensemble draws five times as many samples as the single-chain baseline (10,000 against 2,000), and histogram-based JS distance is biased by sample count, so the comparison was not fair. When we give a single chain the same total budget of 10,000 samples, it beats the ensemble on nine of eleven configurations, ties on the other two, and lowers the mean JS by a further 24%; the ensemble never wins, and the single chain also runs about six times faster. We show, using draws from the true distributions, that the marginal JS estimate of a perfect sampler is almost constant across all targets (0.085–0.087 at 2,000 samples and 0.043–0.044 at 10,000), so the original 18% “gain” was mostly this sample-count effect and not sampler quality. A per-component ablation shows that none of the three mechanisms helps, and a homogeneous ensemble matches the diverse one, so diversity is not the active ingredient either. Finally, when we replace the whole aggregation step with plain uniform pooling of the members’ raw samples, the accuracy goes back to the single-chain level across all eleven configurations (mean JS 0.062 versus 0.065 for the single chain) while the full machinery stays much worse (0.085). This places the entire deficit on the aggregation step, not on pooling. Our conclusion is a cautionary one: for the targets and budgets we study, a single flowMC chain at the full budget is cheaper and more accurate than the ensemble; uniform pooling matches it; and the proposed aggregation mechanisms actively make things worse.

Keywords: Markov chain Monte Carlo; normalizing flows; ensemble sampling; negative results; Jensen–Shannon divergence; finite-sample bias

1 Introduction

Sampling from complex, high-dimensional and multimodal distributions is one of the recurring difficulties in Bayesian inference [5]. Normalizing-flow MCMC, such as flowMC [1,2], combines local

gradient-based moves with a global proposal drawn from a normalizing flow [3,4] that is trained on the running samples. On targets where purely local samplers move slowly, this kind of method usually does well.

When we began this work, our reasoning was the common one: if a single flow-based sampler is good, then a group of them, built with different flow architectures and different step budgets, should be even more reliable, because the weak points of one member could be covered by the others. Ensemble and population MCMC have a long tradition along this line [6,7,8]. So we designed a diverse five-member flowMC ensemble and added three lightweight aggregation mechanisms, each aimed at a failure mode we expected: adaptive tempering for members that explore poorly, coverage-aware reweighting for regions that get missed, and quality-based member weighting so that weaker members count less.

At first the results agreed with our expectation. Across six targets in 20 and 50 dimensions, the ensemble lowered the average marginal JS distance to a reference sample by about 18% relative to a single chain, and on every configuration the difference was statistically significant. We were ready to write this up as a positive method.

The reason this paper is a negative result instead is one control experiment. With the budget allocation we used, called equal-per-member, every member draws as many samples as the single-chain baseline, so five members produce five times the samples in total — 10,000 against 2,000. Histogram-based JS distance is not a fair quantity to compare across different sample sizes, because it carries an upward bias that shrinks as the number of samples grows. The honest comparison is therefore not a single chain at 2,000 samples, but a single chain at the same total of 10,000.

We ran it, and it reversed the conclusion. A single flowMC chain at 10,000 samples is better than the ensemble on nine of eleven configurations, ties on the two remaining ones, and lowers the mean JS by a further 24%. The ensemble never wins. We then checked the metric itself by drawing directly from the true distributions, and found that even a perfect sampler’s marginal JS estimate moves from about 0.085 at 2,000 samples to about 0.043 at 10,000 — a factor close to two, and almost the same number for every target. This accounts for the original “gain” on its own. After that, we kept digging. A homogeneous ensemble of five identical members performs the same as the diverse one, so diversity does nothing here. None of the three aggregation mechanisms helps in the ablation; two of them slightly hurt. And when we throw away the aggregation step entirely and simply concatenate the members’ raw samples, the accuracy returns to the single-chain level. In other words, pooling the members is harmless, but the machinery we put around the pooling is what destroys the quality.

We think this is worth reporting, precisely because the positive version of the story is the easy one to believe and the easy one to publish. The single experiment that breaks it is cheap. The contributions of this paper are:

1. We show that the apparent advantage of a diversity-enhanced flowMC ensemble over a single chain is, for these targets and budgets, an artifact of comparing at unequal sample counts.
2. With a matched-budget control, a single chain matches or beats the ensemble on every configuration (better on nine of eleven, tied on the two banana cases) and is also faster; the ensemble’s aggregation machinery degrades quality rather than improving it, and we localize the loss to the aggregation step by showing that plain uniform pooling recovers single-chain accuracy.
3. We document that marginal JS distance is dominated by finite-sample bias at these sample sizes and is therefore a weak way to compare samplers, which is a small but practical methodological warning.

2 Background and Related Work

2.1 Normalizing-flow MCMC

A normalizing flow writes a distribution as an invertible, learnable transformation of a simple base density, with a tractable Jacobian [3,4]. flowMC [1,2] uses such a flow as a global independence proposal together with local Metropolis-adjusted Langevin steps, and trains a rational-quadratic neural spline flow [4] in alternating training and production phases. In this work we keep flowMC unchanged both for the single-chain baseline and for each ensemble member, so that any difference we see comes from ensembling and aggregation, not from a modified kernel.

2.2 Ensemble and population MCMC

Combining several chains to gain reliability is an old idea, including affine-invariant ensemble samplers [6] and their popular implementation [7], as well as population-based and tempering methods more generally [8,9]. Our setup is different in one respect: the members do not talk to each other while sampling. They are independent flowMC runs and are combined only at the aggregation stage. This isolates the effect of pooling several members from any inter-chain communication.

2.3 Evaluating sample quality

We measure the marginal JS distance [11] between the sampler output and a large reference sample, averaged over the dimensions, together with effective sample size [12] and runtime. A point that becomes central later (Section 5.3) is that histogram-based JS distance has a clear positive finite-sample bias, so a comparison made at different sample counts can mislead.

3 Methods

3.1 Single flowMC baseline

The baseline is one flowMC sampler with two training and two production loops, ten local MALA steps and ten global flow steps per loop, and five training epochs per loop. The flow is a rational-quadratic spline with three coupling layers, eight bins, and two hidden layers of width 64. These settings are kept fixed across all targets. We run this baseline at two sample sizes: 2,000 particles, which was our original baseline, and 10,000 particles, which is the matched control whose total equals the ensemble. We call this single flowMC run the “single-chain” baseline, following MCMC terminology, although each run returns a set of particles rather than one sequential chain.

3.2 Diverse ensemble

The ensemble has five flowMC members that differ in capacity and step allocation: hidden-layer widths $((64, 64), (96, 96), (128, 64), (64, 128), (32, 64, 32))$, number of spline layers (three or four), number of bins (eight to twelve), and the split between local and global steps. The members are independent and are combined only at the aggregation step.

3.3 Aggregation mechanisms

Three mechanisms act at the aggregation stage. **Adaptive tempering** changes a per-member temperature T in $\log \pi_T(x) = \frac{1}{T} \log \pi(x)$, raising T when a member’s ESS ratio is below a target and lowering it otherwise. **Coverage-aware reweighting** clusters the pooled samples on their first

two coordinates with k -means and gives inverse-frequency weights with a boost for rare clusters. **Quality-based member weighting** scores members by

$$s_m = \text{ESS}_m^\beta \exp(-\gamma \text{JS}_m) \exp(-\lambda \sigma_{\log p, m}), \quad (1)$$

and turns the standardized scores into weights with a softmax. Each mechanism was chosen because it targeted a failure mode we believed mattered. As Section 5.4 reports, none of them earns its place.

3.4 Pooling and budget allocation

The pooled samples are combined by multiplying the member weights and the coverage weights, normalizing, and **resampling with replacement** to a target count of (per-member size) $\times$ (number of members). Under **equal-per-member**, each member draws 2,000 samples, so the ensemble total is 10,000. Under **equal-total**, the 2,000-sample budget is split across members (400 each). The equal-per-member ensemble therefore uses five times the compute and produces five times the samples of the 2,000-particle single chain, and the fair comparison is the 10,000-particle single chain.

4 Experimental Setup

4.1 Targets and protocol

We use six targets that stress different weaknesses of a sampler: a banana density (strong nonlinear correlation), a bimodal Gaussian (separated modes), Neal’s funnel [10] (varying scale), a correlated Gaussian (correlation 0.95), the Rosenbrock density (curved valley), and a Gaussian ring (eight modes on a ring). All are run in 20 and 50 dimensions except the ring, which is 20D only, giving eleven configurations. Each one is repeated five times with deterministic seeds, the single-chain and ensemble runs in a repetition share a seed stream, and the reference samples (20,000 per target) are drawn from the generative form of each target. All seeds, configurations and code are released [13].

4.2 Metrics

The main metric is the average marginal JS distance computed on 100-bin histograms. We also report ESS from the integrated autocorrelation and the wall-clock runtime. The ESS we report is computed on the returned particle or pooled sample array and serves only as a diagnostic of duplication and correlation in the returned sample set; it should not be read as a full trajectory-based MCMC mixing ESS. Where useful we give paired t -tests and Cohen’s d over the five repetitions.

5 Results

5.1 The apparent ensemble gain

Against the 2,000-particle single chain, the equal-per-member ensemble lowers the marginal JS distance on all eleven configurations, by 6.8% to 29.2% (mean 18.2%), and every configuration is significant at $p < 0.05$. Read on its own, this looks like a clear success for ensembling, and the largest improvements are on the multimodal and structured targets. The rest of this section explains why it is not what it looks like.

5.2 The equal-budget control

The equal-per-member ensemble draws 10,000 samples in total, while the baseline it was compared with draws only 2,000. The fair comparison is a single chain at 10,000 samples, which we give in Table 1 and Figure 1. The single 10,000-particle chain reaches a mean JS of 0.065, against 0.085 for the ensemble and 0.104 for the 2,000-particle baseline. It beats the ensemble on nine of eleven configurations, ties on the two banana cases (within repetition noise), with 17% to 44% improvements on exactly those targets where the ensemble had looked strongest (bimodal, funnel, correlated, ring).

The single-chain advantage is not marginal where it exists: the per-configuration gap is several to over a hundred times the single-chain repetition standard deviation. The two banana cases are the exception, and they are informative rather than awkward. There, both the single chain and the ensemble sit far above the noise floor (JS around 0.10–0.14 against a floor of 0.043), which tells us that flowMC is simply failing to capture the banana’s strong nonlinear correlation, no matter the sample count or the ensembling. This is a sampler-limited regime, not a sample-limited one, and the near-tie reflects that neither extra samples nor extra members help. So the banana is not a case where ensembling succeeds; it is a case where nothing we tried does.

Table 1: Marginal JS distance (mean of five repetitions). The equal-budget single chain on the right is the proper control for the ensemble in the middle. Lower is better. “Tie” marks differences within repetition noise.

Target	Dim	Single @2,000	Ensemble @10,000	Single @10,000	Single@10k vs ens
Banana	20	0.169	0.138	0.137	tie
Banana	50	0.127	0.095	0.097	tie
Bimodal	20	0.087	0.061	0.045	−27%
Bimodal	50	0.086	0.061	0.045	−27%
Correlated	20	0.087	0.081	0.045	−44%
Correlated	50	0.086	0.071	0.044	−38%
Funnel	20	0.086	0.074	0.044	−40%
Funnel	50	0.087	0.079	0.045	−43%
Ring	20	0.097	0.071	0.059	−17%
Rosenbrock	20	0.129	0.114	0.090	−20%
Rosenbrock	50	0.103	0.089	0.062	−30%
Mean		0.104	0.085	0.065	−24%

5.3 Why: marginal JS distance is saturated by finite-sample bias

The reason behind the reversal is simple, and we should have checked it earlier. Histogram-based JS distance is a biased estimator: it reports a larger divergence when the sample is small, and the bias goes down as the sample grows. Drawing directly from the true distributions, we find that the marginal JS estimate of a perfect sampler is remarkably constant across all eleven configurations: 0.085–0.087 at 2,000 samples and 0.043–0.044 at 10,000, a factor close to two (about 1.95–2.0) regardless of the target or the dimension. The floor is set by the number of histogram bins and the sample size, not by the target. The 2,000-particle single chain sits at about 0.104 on average; on the Gaussian-like targets it is already at its 2,000-sample floor (around 0.086), with the mean pulled up by the banana and Rosenbrock cases where flowMC itself struggles. So on the targets where the

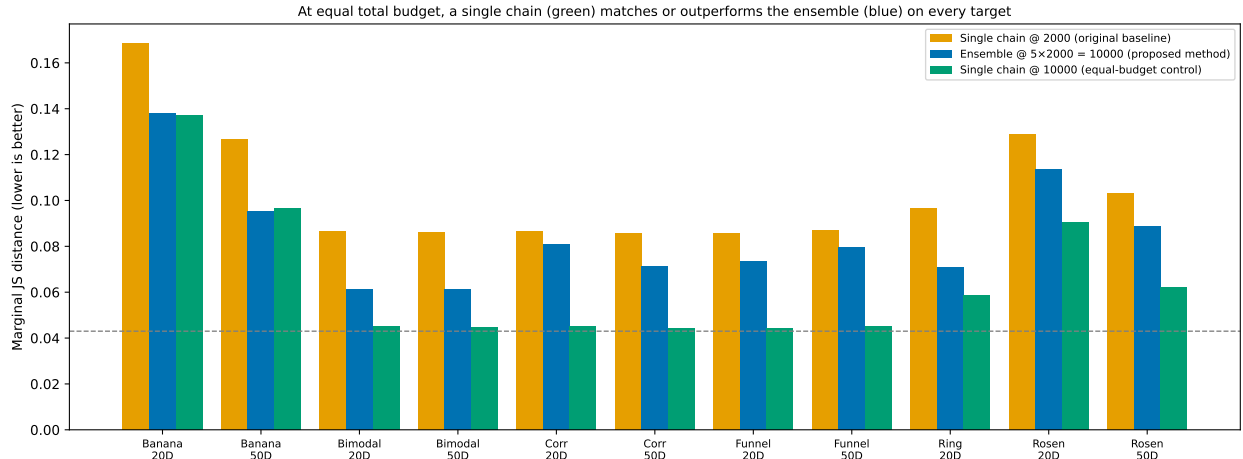


Figure 1: At equal total budget, the single 10,000-particle chain (green) matches or outperforms the ensemble (blue) on every target, winning on nine of eleven configurations and tying on the two banana cases. The ensemble’s apparent advantage over the 2,000-particle baseline (orange) does not survive the matched comparison. The dashed line is the finite-sample noise floor of a perfect sampler at 10,000 samples (Section 5.3).

metric can resolve sampler quality, the single chain already reproduces the marginals nearly as well as it can see at that sample size. So the original 18% “gain” of the ensemble is for the most part just the difference between the 2,000-sample and 10,000-sample floors.

Figure 2 makes this concrete and also previews the ensemble’s problem. The single chain at 2,000 samples sits just above the 2,000-sample floor, and the single chain at 10,000 sits just above the 10,000-sample floor. Both of them are near-optimal for the metric at their sample size. The full ensemble, although it nominally produces 10,000 samples and its measured effective sample size is close to that (about 9,800 by integrated autocorrelation), still lands at a JS value (0.085) that a single chain reaches with only about 2,000 samples. So the ensemble is not short of effective samples; something in the way they are aggregated keeps the histogram from improving the way more samples should. We return to what in Section 5.5.

Two things follow. First, the comparison in Section 5.1 was not informative, because it placed the two samplers at different points on the bias curve. Second, marginal JS distance is a weak way to tell samplers apart in this regime, and any benchmark using it should either match the sample counts exactly or report the floor. We learned this the hard way.

5.4 Ablation: the aggregation mechanisms do not help

We turn off each mechanism in turn (equal-per-member) and compare to the full ensemble. None of them gives a net benefit. Removing coverage-aware reweighting improves the mean JS by 5.4% (best on the correlated Gaussian, where it improves JS by 22% in 20D); removing adaptive tempering improves it by 4.2%; removing member weighting changes it by under 1%. The full method is the worst of the four ensemble variants. A homogeneous ensemble of five identical members performs essentially the same as the diverse one (mean JS 0.0840 versus 0.0849, a difference inside the repetition noise), so member diversity is not the active ingredient either. This was more negative than we expected when we designed the mechanisms, and together with Section 5.2 it says clearly that neither diversity nor the aggregation machinery earns its place. These are one-at-a-time ablations rather than a full factorial over all mechanism combinations. Figure 3 summarizes these

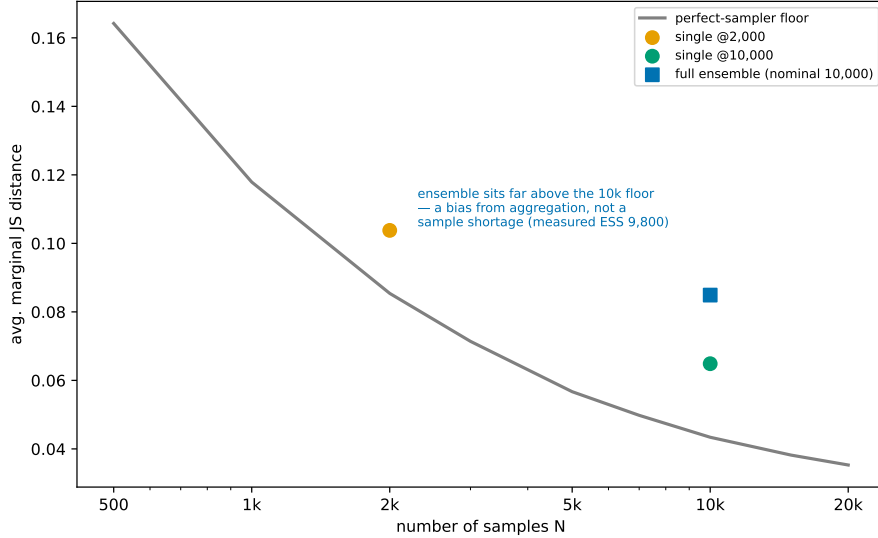


Figure 2: The marginal JS noise floor of a perfect sampler falls with the number of samples (grey curve, averaged over the correlated, bimodal and funnel targets; the same sample-size dependence holds across all eleven configurations). The single chains at 2,000 and 10,000 samples sit close to their respective floors, while the full ensemble, plotted at its nominal 10,000 samples, sits far above the 10,000-sample floor. Its marginal-JS value is comparable to the 2,000-sample regime under this diagnostic even though its measured sample-set ESS is near 10,000, so the gap is a bias introduced by the aggregation rather than a shortage of samples.

comparisons.

Table 2: Ablation under the equal-per-member budget: mean marginal JS distance over the eleven configurations. Lower is better. All four ensemble variants are still beaten by the single 10,000-particle chain at 0.065.

Variant	Mean JS	vs. single @2,000	vs. full ensemble
Single chain @2,000	0.1038	—	—
Full ensemble	0.0849	−18.2%	—
No coverage reweighting	0.0803	−22.6%	−5.4%
No adaptive tempering	0.0813	−21.7%	−4.2%
No member weighting	0.0842	−18.9%	−0.8%
Single chain @10,000	0.0649	−37.5%	−23.6%

5.5 Localizing the cause: uniform pooling versus the aggregation machinery

The matched-budget control tells us the full ensemble loses, but it does not tell us which part of the ensemble is responsible. To find out, we replace the aggregation (the weighted resampling-with-replacement and the coverage, tempering and weighting mechanisms) with the simplest possible alternative: we just concatenate the five members’ raw 2,000-sample outputs into one pool of 10,000 unique samples, with no resampling and no weights. Across all eleven configurations, this uniform pool matches the single 10,000-particle chain (mean JS 0.062 against 0.065) and sits far below the

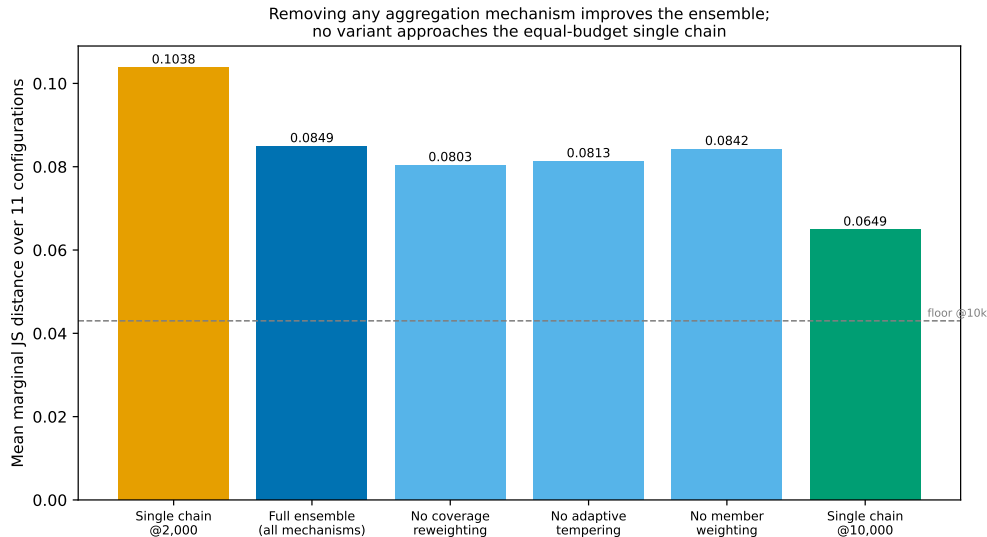


Figure 3: One-at-a-time ablation under the equal-per-member budget (mean marginal JS over the eleven configurations). Removing any of the three mechanisms improves the ensemble, so the full method is the worst of the four variants, and no ensemble variant approaches the equal-budget single chain. The dashed line is the 10,000-sample noise floor.

full ensemble (0.085); it matches the single chain on every target within repetition noise, and is slightly better on the two hardest ones, the banana and the ring, where pooling several independent runs seems to help a little (Table 3). We do not over-read this small edge, since we did not separate it from a plain multi-start effect, but the main point is clear: pooling does not hurt.

So pooling the members is harmless on its own: a uniform pool of the same five members reaches the same accuracy as a single chain at the same budget. The whole deficit of the full ensemble comes from the aggregation machinery, not from the act of pooling. The mechanism is bias, not lost samples. The full ensemble and the uniform pool have almost the same measured effective sample size (about 9,800 and 9,700 respectively, both near the nominal 10,000), yet their accuracy is far apart, so the deficit cannot be a shortage of effective samples. What differs is that the aggregation reweights the pooled samples by coverage and member quality and resamples from those weights, which pulls the sample distribution away from the target and biases the marginals; the uniform pool instead preserves the empirical distribution produced by the member samplers without imposing additional aggregation weights. The ablation supports this directly: removing the coverage and tempering reweighting is what improves the JS (Section 5.4). We cannot, from the logged summaries alone, separate how much of the bias comes from the resampling-with-replacement and how much from the reweighting, so we attribute it to the aggregation step as a whole.

This control also closes a confound that we worried about. One might suspect that the single 10,000-particle chain wins only because its flow is trained on more data (10,000 samples) than each ensemble member (2,000). But in the uniform pool, every member is still trained on only 2,000 samples, exactly like the ensemble members, and the pool of these members already matches the single chain. So the matched single-chain accuracy appears to be driven mainly by the total returned sample count under this marginal-JS diagnostic, not by how much data any individual flow was trained on. And the objection that the ensemble just needs a better-implemented pooling step does not hold either: the simplest pooling we could write already matches the single chain, and no aggregation we tested beats it.

Table 3: Marginal JS distance (mean of five repetitions) across all eleven configurations. Plain uniform pooling of the members recovers single-chain accuracy; the full aggregation machinery does not. Lower is better.

Target	Dim	Full ensemble	Uniform pool	Single @10,000
Banana	20	0.138	0.128	0.137
Banana	50	0.095	0.084	0.097
Bimodal	20	0.061	0.044	0.045
Bimodal	50	0.061	0.044	0.045
Correlated	20	0.081	0.044	0.045
Correlated	50	0.071	0.044	0.044
Funnel	20	0.074	0.044	0.044
Funnel	50	0.079	0.045	0.045
Ring	20	0.071	0.055	0.059
Rosenbrock	20	0.114	0.090	0.090
Rosenbrock	50	0.089	0.062	0.062
Mean		0.0849	0.0622	0.0649

5.6 Equal-total budget

A control on the other side splits the 2,000-sample budget across the five members (400 each), so the ensemble and the single chain produce the same total of samples. Here the ensemble is worse on all eleven configurations, by 11% to 64%, with large negative effect sizes. Members that are starved of samples train weaker flows, and pooling them does not recover the quality of one chain that keeps the full budget. Together with Section 5.2 this brackets the conclusion from both directions: at an equal total of samples the ensemble loses whether the budget is small (2,000) or large (10,000). Figure 5 shows the per-configuration comparison.

5.7 Cost

On a single accelerator the equal-per-member ensemble takes about five times the wall-clock time of the single chain, because it runs five separate flowMC instances. The cost of one flowMC run is dominated by its fixed training loops, not by the particle count: on our hardware the 10,000-particle chain runs in about 15 seconds, comparable to the 2,000-particle chain (about 18 seconds) and roughly six times faster than the ensemble (about 92 seconds). The single chain therefore gets five times the samples at almost no extra cost, while the ensemble pays a fivefold cost for samples that are worse. (With five accelerators the members could run in parallel and the wall-clock would become comparable, but at five times the hardware and still with worse samples.) In none of our settings is the ensemble the better choice. Figure 6 places the three samplers on the accuracy–cost plane.

6 Discussion

6.1 What actually happened

Several findings point to one conclusion. The ensemble’s apparent advantage over a small single chain is a sample-count artifact (Section 5.3); at an equal total budget a single chain is better

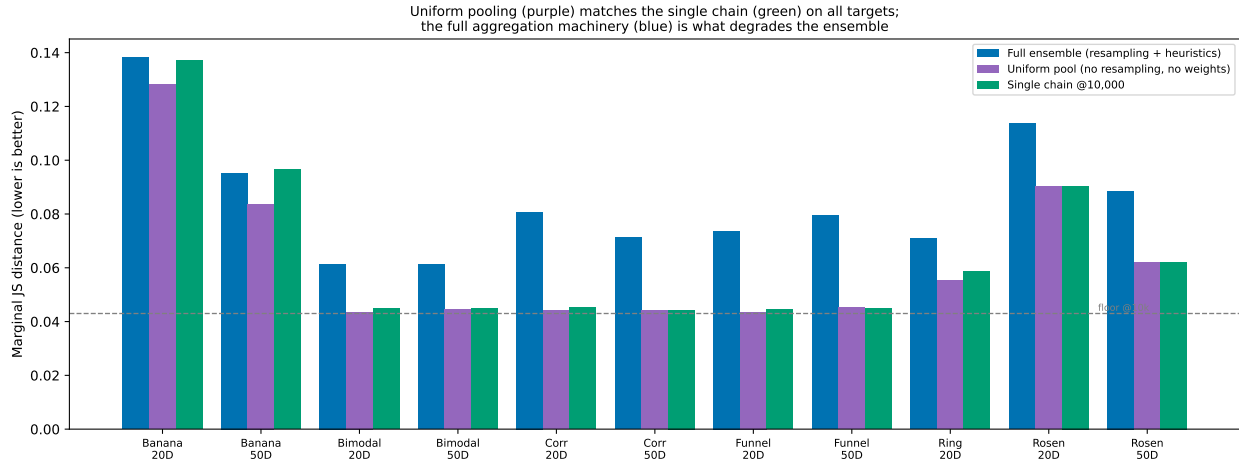


Figure 4: Across all eleven configurations, a plain uniform pool of the five members’ raw samples (purple) reaches the same accuracy as a single chain at the matched budget (green), while the full aggregation machinery (blue) stays far worse. Pooling is harmless; the machinery around it is not.

(Sections 5.2 and 5.6); member diversity does nothing (Section 5.4); and none of the aggregation heuristics helps (Section 5.4). The direct control in Section 5.5 then locates the cause: when we replace the aggregation with plain uniform pooling, the accuracy returns to the single-chain level, so the deficit comes entirely from the aggregation machinery and not from pooling. The mechanism is bias rather than lost samples: the ensemble’s measured effective sample size stays near its nominal 10,000, almost the same as the uniform pool’s, yet its JS is far worse, so the coverage and tempering reweighting and the weighted resampling distort the marginals away from the target. A uniform pool preserves the members’ empirical outputs without adding another weighting step, which is why it matches the single chain under this marginal-JS evaluation.

Looking back, the part that is a little uncomfortable to admit is how convincing the wrong version looked. The first draft of this work reported the 18% improvement with significance on every target and large effect sizes, and all of that was technically correct — it just answered the wrong question. The lesson we take from it is mundane but easy to forget under the pressure of a positive result: before believing a comparison, make the baselines spend the same budget.

6.2 Implication for practice

For the targets and budgets we study, the practical advice is the opposite of the method we set out to build. Given a fixed sampling budget, run one flowMC chain at the full budget instead of splitting or multiplying it across an ensemble. If for some reason the members are pooled, pool them uniformly and do not expect to beat the single chain. And avoid coverage-style reweighting and diagnostic-driven tempering, which here only lower the quality. Ensembling in this setting adds compute and complexity and gives nothing back. A third, more general guideline concerns evaluation rather than sampling: when a histogram-based divergence such as marginal JS is used to compare samplers, the comparison is only meaningful at matched sample counts, and reporting the finite-sample floor of the metric alongside the results makes the remaining headroom visible at a glance.

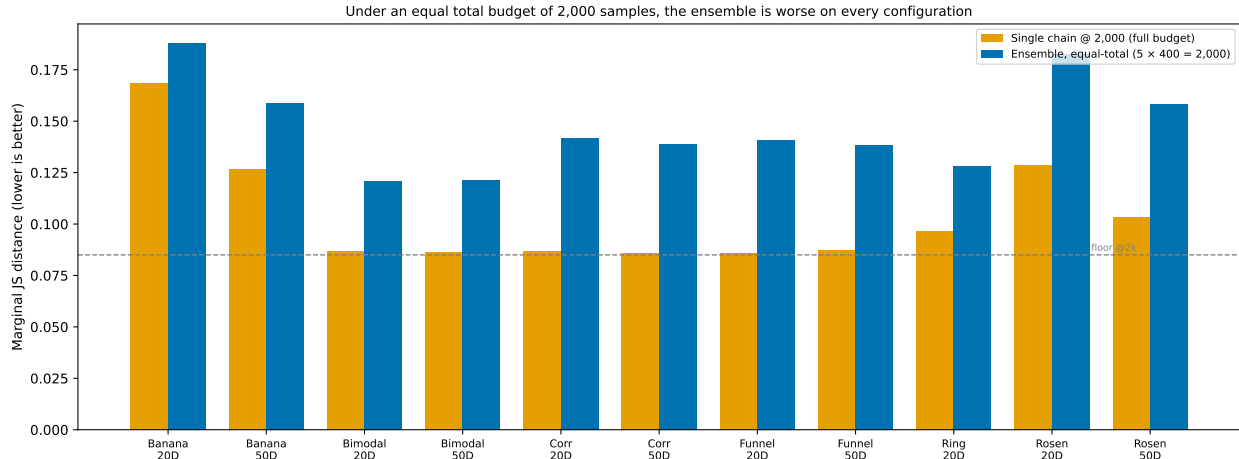


Figure 5: Equal-total budget: with the same 2,000-sample total, the ensemble (five members of 400 samples each) is worse than the single chain on every configuration, by 11% to 64%. The dashed line is the 2,000-sample noise floor.

6.3 Scope and limitations

Our conclusion concerns marginal JS distance on these synthetic targets. We measured only marginal, not joint, distributional error, and a joint metric such as energy distance or maximum mean discrepancy could show mode-coverage effects that marginal JS cannot. This caveat is not the same for every target: for the bimodal Gaussian, whose modes are separated along a single axis, the marginal JS on that axis does reflect mode coverage, so our conclusion there is on firmer ground; for the Gaussian ring, whose eight modes sit in the plane, the marginals are partly washed out and a chain that covered only some modes could still match them, so the single-chain result on the ring should be read as a statement about marginal accuracy and not about mode coverage. A joint metric is the natural way to settle whether ensembling helps for genuinely two-dimensional multimodal structure. We also used the aggregation mechanisms at fixed default hyperparameters without per-target tuning; tuning might change their contribution, but a mechanism that has to be tuned carefully just to avoid harming the result is of limited practical value. Finally, we tested six synthetic targets at 20 and 50 dimensions, one flow family, and one ensemble size (five members); real posteriors, other flows, or other ensemble sizes may behave differently. Beyond these caveats, three extensions would sharpen the picture: evaluating the same controls under a joint metric such as energy distance or maximum mean discrepancy, repeating the study on real posteriors with funnel-like or multimodal geometry, and varying the ensemble size to test whether the aggregation-induced bias grows with the number of members.

7 Conclusion

We set out to validate a diversity-enhanced flowMC ensemble, and instead we found that its apparent benefit was an artifact of comparing at unequal sample counts. With a matched 10,000-sample budget a single flowMC chain matches or beats the full ensemble on every target — winning on nine of eleven configurations and tying on the two banana cases — and lowers the mean marginal JS distance by 24%, while running about six times faster. Taking the ensemble apart shows where the benefit went: diversity does nothing, none of the three aggregation heuristics helps, and replacing

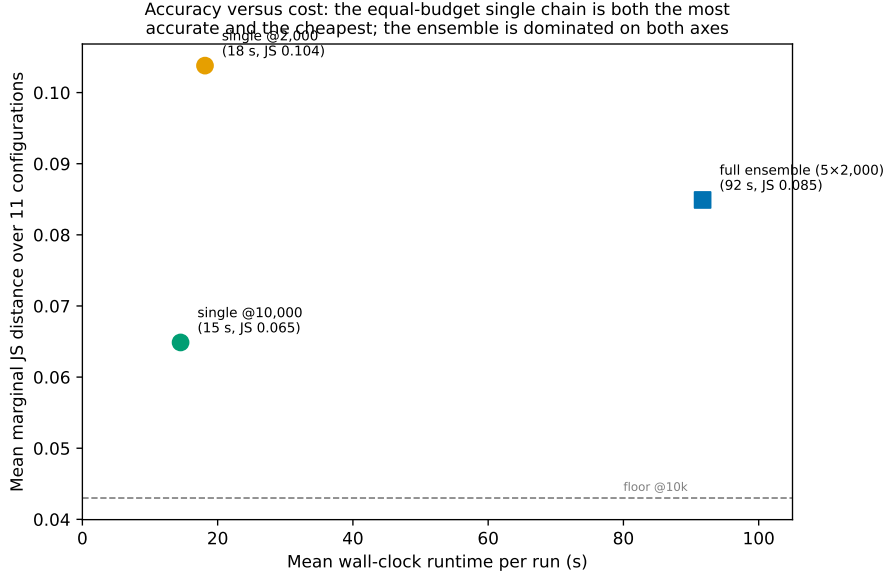


Figure 6: Accuracy versus cost (mean over the eleven configurations; measured wall-clock runtimes). The equal-budget single chain is simultaneously the most accurate and the cheapest option, so the ensemble is dominated on both axes: it costs about six times more and is less accurate.

the aggregation with plain uniform pooling recovers the single-chain accuracy. The deficit therefore comes entirely from the aggregation machinery. The coverage and tempering reweighting and the weighted resampling bias the marginals, even though the ensemble’s measured effective sample size stays near its nominal count, while the metric used to claim the original gain is saturated by finite-sample bias. The recommendation is short: run one flowMC chain at the full budget; if you pool members, pool them uniformly and expect no gain over a single chain; and stay away from the resampling-and-reweighting machinery, which only hurts for the targets and the marginal-JS evaluation we study. We release the controls and code so that others can check this. The broad lesson is an old one that still catches us: an ensemble compared against a smaller baseline can look effective for reasons that have nothing to do with the ensemble. The remedy is equally simple: match the budgets, know the finite-sample floor of the metric, and give a plain control the chance to falsify the method early.

Reproducibility

All target definitions, member configurations, per-repetition seeds, the control scripts (single at 10,000, the homogeneous ensemble, and the uniform-pooling control) and the analysis code are available at [13]. Every number reported here can be regenerated from the released seeds. The repository also includes the CSV outputs used to produce Tables 1–3 and Figures 1–3.

References

- [1] K. W. K. Wong, M. Gabri  , D. Foreman-Mackey. flowMC: Normalizing flow enhanced sampling package for probabilistic inference in JAX. *Journal of Open Source Software*, 8(83):5021, 2023.

- [2] M. Gabrié, G. M. Rotskoff, E. Vanden-Eijnden. Adaptive Monte Carlo augmented with normalizing flows. *PNAS*, 119(10):e2109420119, 2022.
- [3] G. Papamakarios, E. Nalisnick, D. J. Rezende, S. Mohamed, B. Lakshminarayanan. Normalizing Flows for Probabilistic Modeling and Inference. *JMLR*, 22(57):1–64, 2021.
- [4] C. Durkan, A. Bekasov, I. Murray, G. Papamakarios. Neural Spline Flows. *NeurIPS* 32, 7509–7520, 2019.
- [5] S. Brooks, A. Gelman, G. L. Jones, X.-L. Meng (eds.). *Handbook of Markov Chain Monte Carlo*. Chapman & Hall/CRC, 2011.
- [6] J. Goodman, J. Weare. Ensemble samplers with affine invariance. *Comm. App. Math. Comp. Sci.*, 5(1):65–80, 2010.
- [7] D. Foreman-Mackey, D. W. Hogg, D. Lang, J. Goodman. emcee: The MCMC Hammer. *PASP*, 125(925):306–312, 2013.
- [8] D. J. Earl, M. W. Deem. Parallel tempering: Theory, applications, and new perspectives. *Phys. Chem. Chem. Phys.*, 7(23):3910–3916, 2005.
- [9] R. M. Neal. Sampling from multimodal distributions using tempered transitions. *Statistics and Computing*, 6(4):353–366, 1996.
- [10] R. M. Neal. Slice sampling. *The Annals of Statistics*, 31(3):705–767, 2003.
- [11] J. Lin. Divergence measures based on the Shannon entropy. *IEEE Trans. Inf. Theory*, 37(1):145–151, 1991.
- [12] C. J. Geyer. Practical Markov Chain Monte Carlo. *Statistical Science*, 7(4):473–483, 1992.
- [13] M. Shi. flowmc-ensemble: code and reproducibility artifacts. GitHub, <https://github.com/mingyus3/flowmc-ensemble>, 2025.